

Project Proposal: FreshTrack Wholesale ERP

1. Member Information

- **Member 1:** Chan Long Him, 3036219035
- **Member 2:** Katarina Jan Jones, 3036712374
- **Member 3:** Ma Tsz Hin, 3036104242
- **Member 4:** Tse Chun Hong, 3036225199

2. Project Title

FreshTrack: Wholesale Food ERP & Order Management System

3. Project Description

FreshTrack is a comprehensive wholesale management platform designed for food suppliers to handle B2B (business-to-business) sales and internal inventory operations seamlessly. The target end-users are wholesale buyers (e.g., local restaurant owners) who need a reliable website to purchase bulk food ingredients, and wholesale administrative staff who require an ERP dashboard to monitor livestock levels, process incoming orders, and track perishable food expiration dates.

4. Feature List

Must-have (Core Features)

- **Role-Based Authentication:** Distinct login workflows and access permissions for wholesale buyers versus administrative staff.
- **Product & Inventory Catalog:** A responsive storefront allowing buyers to browse food items, view live inventory levels, and see wholesale pricing.
- **Shopping Cart & Checkout System:** A portal for buyers to submit bulk orders, which will automatically deduct the requested item quantities from the central database upon checkout.

- **Admin ERP Dashboard (with Expiry Tracking):** A secure backend interface where staff can add new food shipments, update stock quantities, and log expiration dates for perishable goods.
- **Order Management:** A history view for buyers to track their purchases, and a control panel for admins to update order statuses (e.g., Processing, Shipped, Delivered).

Nice-to-have (Bonus Features)

- **Automated Expiry Alerts:** A dashboard widget that automatically flags food batches expiring within the next 14 days.
- **Data Visualization:** Interactive charts displaying sales trends and inventory distribution in the admin dashboard.
- **OAuth 2.0 Integration:** Allowing users to log in securely using third-party providers like Google or GitHub.

5. Technology Stack

- **Frontend:** React Single Page Application (SPA)
- **Backend:** Node.js, Express & cors (middleware)
- **Database:** MySQL
- **Deployment:** Docker Compose on HKU's Linux virtual machine

6. Preliminary Team Task Allocation

- **MA TSZ HIN:** Database & Deployment. Responsible for designing the MySQL schema and managing the Docker container configurations.
- **Katarina Jane Jones:** Backend API. Responsible for building the Node.js/Express RESTful API routes and writing the SQL queries.
- **Tse Chun Hong:** Frontend UI (Buyer Storefront). Responsible for building the responsive SPA interface for the shopping cart and product catalog.
- **Chan Long Him:** Frontend UI (Admin Dashboard). Responsible for the ERP dashboard components and connecting frontend pages to the backend REST API via fetch requests.

7. Problem & Target User Context

Wholesale food suppliers often struggle to manage the dual complexities of B2B client ordering and perishable inventory tracking. Currently, many suppliers rely on phone orders and manual spreadsheets, leading to overselling or spoiled stock. Our target users are restaurant managers who need a quick, 24/7 web-based ordering portal, and warehouse staff who need a centralized system to fulfill those orders while strictly monitoring food expiration dates. A web application is the perfect fit because it provides real-time database synchronization across all users without requiring client-side installations.

8. High-level Workflow Description

1. A wholesale buyer logs into the platform, browses the available food catalog, and adds bulk items to their cart.
2. The buyer proceeds to checkout, confirming their delivery details, at which point the system reserves the stock.
3. Simultaneously, an admin logs into the ERP dashboard and sees the new order under the "Processing" tab.
4. The admin reviews the required stock, picks the food items with the closest expiration dates, and marks the order as "Shipped".

9. Anticipated Learning Challenges & Self-assessment

1. **Challenge 1: Steep Learning Curve with the Full-Stack Transition**
 - a. Because our team's background is primarily in Python, adopting a completely new JavaScript ecosystem (React SPA, Node.js, Express) alongside MySQL and Docker deployment presents a heavy initial learning curve.
 - b. **Practical Plan:** We will dedicate our initial project time strictly to foundational learning rather than feature building. We will rely on official documentation (like react.dev) to understand component state, and we will build small, isolated test scripts before integrating new technologies into the main FreshTrack repository.
2. **Challenge 2: Cross-Functional Integration and Workflow Coordination**
 - a. Since our tasks are highly specialized with distinct owners for the database, backend API, buyer storefront, and admin dashboard, ensuring these

separate pieces communicate flawlessly (e.g., the React frontend correctly passing fetch requests to the Express backend) will be difficult to coordinate.

- b. **Practical Plan:** To prevent the common issue of code working locally in isolation but failing when combined, we will hold mandatory, twice-weekly live integration syncs. During these sessions, the frontend and backend developers will test API endpoints together in real time, ensuring CORS configurations, route names, HTTP request structures, and JSON data formats align before merging into the main branch.

3. Challenge 3: Managing Time Limits Against Complex Debugging

- a. Balancing the project's Alpha and Beta deadlines while troubleshooting unfamiliar errors in a new tech stack (such as routing issues in React or database connection errors in Docker) poses a significant risk to our timeline.
- b. **Practical Plan:** We will strictly enforce our scope by focusing entirely on the "Must-have" core features first, completely ignoring the "Nice-to-have" features until the baseline is stable. We will also establish a rule that if any member is stuck on a bug for more than two hours, they must escalate it to the group to avoid wasting development time.